Last Updated May 4, 2026

Frontend Architecture Guide: UCSC Finance & Procurement Dashboard

This document provides a technical breakdown of the React and TypeScript frontend powering the campus finance and procurement dashboard. It details the state management, data pipeline, and component architecture required to maintain and expand the system.

1. Architectural Overview: Dual Paradigms

The frontend handles two distinct types of supply chain workflows, utilizing two different architectural paradigms to prevent UI latency and manage complexity:

A. The "What-If" Sandbox (Finance & Expense Auditing)

Used for CruzBuy, Amazon, and OneCard data. The frontend manages local merging. It stacks temporary, projected data parsed from user-uploaded CSVs on top of historical baseline data fetched from the API.

B. Action-Oriented Triage (Predictive Procurement)

Used for the Campus Store inventory. Instead of manual projections, this paradigm relies on BigQuery ML (`ARIMA_PLUS`). The backend calculates strict supply chain math (Reorder Points, Suggested Orders), and the frontend's job is purely to render this data safely using **Server State Management** via `@tanstack/react-query` to cache aggressive, data-heavy ML queries.

2. Core State Management

Local State Controller (`Dashboard.tsx`)

Serves as the central controller for the Finance side of the app.

- **Passive Cache:** `liveRawTopItems`, `liveRawSpend`, `projectedData` (Raw API/CSV data).
- **Active Display:** `topItems`, `rawSpendSeries` (Transformed arrays passed to UI).

- **Temporal Filters:** `spendRangeMode`, `selectedQuarter` (Controls the X-axis time window).

Server State Controller (`InventoryInsights.tsx`)

Because the ML model predictions are heavy and complex, we abandon local `useState` arrays in favor of `@tanstack/react-query`.

- **Query Key Caching:** The `['bookstore-insights', timePeriod]` array caches the ML output. If a user toggles from "Next 30 Days" to "Next 90 Days" and back, the UI loads instantly from RAM without hitting the FastAPI backend again.

- **Stale Time:** Set to 30 minutes (`1000 * 60 * 30`) to prevent redundant BigQuery costs while ensuring the procurement team is looking at fresh data each session.

3. The Data Pipelines

Pipeline A: The Local Merging Pipeline (CSV Projections)

Triggered by `useEffect` and `useMemo` hooks in `Dashboard.tsx`.

1. **Merging Top Items:** Aligns historical metrics with pending CSV metrics (`projected_count` / `projected_spent`).

2. **Building Spend Series:** Injects `pending_spend` directly into the matching month period of the historical time-series array.

3. **Dynamic Item Filtering:** A heavy `useMemo` block filters the unified array based on Search Query, selected Category, and Financial Min-Spend gates.

Pipeline B: The Defensive Display Pipeline (ML Insights)

The backend does the heavy procurement math (Safety Stock, ROP, Target). The frontend pipeline in `InventoryInsights.tsx` is dedicated to **Defensive Display Engineering**—ensuring volatile data never breaks the UI.

1. **Floor Clamping:** All incoming integers (`current_stock`, `reorder_point`, `predicted_demand`) are clamped using `Math.max(0, value)` to prevent negative values from crashing CSS.

2. **Dynamic Scale Calculation:** The `maxScale` variable dynamically finds the highest value (whether it's an upper bound spike, or an extreme overstock) and adds a 15% padding so visualizer bars never overflow their containers.

3. **Position Mapping:** Converts the raw clamped values into percentages (0% to 100%) to perfectly align the visualizer dots and threshold markers within the Material You cards.

4. Component Deep Dive

`InventoryInsights.tsx` (New)

The high-density triage dashboard for ML predictions.

- **Micro-Cards:** Uses Material Design 3 (MD3) principles (tonal surfaces, 24px border radii) to display complex ML data cleanly.

- **The Action Row:** Dynamically renders UI badges based on backend statuses (`Critical Reorder`, `Dead Stock Risk`, `Monitor Closely`).

- **Visualizer Bar:** A highly compacted, CSS-driven gauge showing the exact position of current stock relative to the Target prediction and the Reorder Point (ROP).

- **AI Context Popovers:** Implements `shadcn/ui` popovers to hide verbose AI reasoning strings behind an interactive "Analysis" trigger, saving critical screen real estate.

`ProjectionUploader.tsx`

Handles CSV file ingestion and API communication for staging.

- **Smart Detection:** Uses the `FileReader` API to scan headers for signature columns, automatically determining the dataset type (Amazon vs. CruzBuy) to prevent user routing errors.

`TopItemsChart.tsx` & `TransactionsOverTimeChart.tsx`

Recharts implementations focused on historical and projected spend.

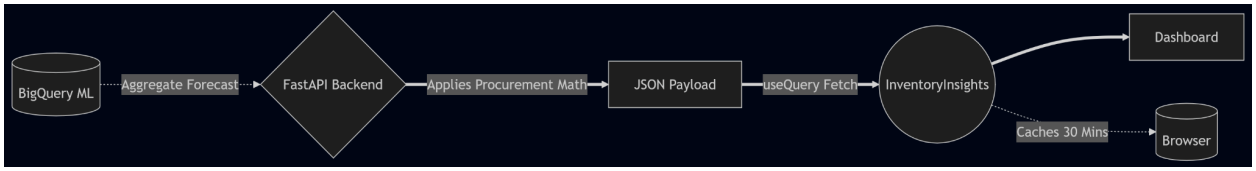
- **Stacked Bars:** Utilizes `stackId="a"` to draw the baseline historical data and seamlessly stack the pending CSV projection data directly on top.

`DatasetExplorer.tsx`

A dense data grid for exact auditing of cleaned CSV outputs.

- **Tooltip Context:** Wraps the entire view in a `TooltipProvider` to support accessible, delayed-hover context for complex data cells.

System Diagram



Last updated 05/04/2026